

XCS: XRPL Credential Service Protocol

Thomas Hussenet thomas@xrpl-commons.org

Protocol Specification v1.0
October 30, 2025

Abstract

XCS (XRPL Credential Service) is a lightweight coordination protocol built on top of the XRP Ledger’s native Credentials amendment. XCS provides a shared namespace for credential schemas and manages the credential lifecycle on-chain, while deliberately delegating storage, privacy, verification strategies, and governance to applications. By adopting a minimalist design—the protocol coordinates, applications innovate—XCS establishes core primitives for schema registration, credential lifecycle management, and composable data structures. This specification describes the protocol layer and its design rationale. A forthcoming technical specification will detail error codes, precise state transitions, and implementation requirements.

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Design Approach	3
2	Design Philosophy	3
2.1	Core Principles	3
2.2	Coordination Layer Philosophy	4
2.2.1	Protocol Responsibilities (On-chain)	4
2.2.2	Application Responsibilities (Off-chain)	4
3	Protocol Specification	5
3.1	Data Types	5
3.2	Schema Definition	5
3.2.1	Schema Structure	5
3.2.2	Schema Registration	5
3.2.3	UID Computation	6
3.2.4	Schema Immutability and Versioning	6
3.3	Schema Inheritance	7
3.3.1	Multiple Inheritance	7
3.3.2	Selective Field Exclusion	7
3.3.3	Inheritance Rules	8
3.4	Protocol Operations	9
3.4.1	Two-Layer Architecture	9
3.4.2	XCS Core Operations and Memo Prefixing	9
3.4.3	Schema Registration	10

3.4.4	Credential Issuance	10
3.4.5	Credential Acceptance	10
3.4.6	Credential Revocation	10
3.5	Credential Metadata Standard	11
3.5.1	Metadata Purpose	11
3.5.2	Metadata Structure	11
3.5.3	Metadata Application	12
3.6	XCS Indexer	12
3.6.1	Responsibilities	12
3.6.2	Deterministic Output	12
3.7	Cryptographic and Trust Model	13
3.7.1	Inherited Security from XRPL	13
3.7.2	XCS-Specific Guarantees	13
3.8	Credential Storage and Verification	13
3.8.1	Verification Flow	14
4	Illustrative Application	14
4.1	XRP Name Service: Decentralized Profiles	14
5	Security Considerations	15
5.1	Schema Poisoning	15
5.2	Storage Availability	15
5.3	Privacy and Relationship Disclosure	15
5.4	Type Validation	15
6	Future Work	16
6.1	Protocol Extensibility	16
6.2	Ecosystem Development	16
7	Conclusion	16
A	Verification Pseudocode	17
B	Example Schema Definitions	19
B.1	Simple KYC Credential	19
B.2	Composable Identity Schema	19
B.3	Gaming Achievement Credential	19

1 Introduction

1.1 Problem Statement

The digital credentials ecosystem lacks a decentralized coordination layer. Currently, credential systems are siloed within proprietary platforms, making it impossible for credentials issued by one party to be recognized and verified by another without custom integration. This fragmentation creates vendor lock-in, prevents interoperability, and forces applications to duplicate infrastructure.

The XRP Ledger's Credentials amendment provides native support for credential issuance, acceptance, and revocation on-chain. However, it lacks a standardized way to define, discover, and reference credential schemas. Without this coordination layer, each application must define its own schema conventions, leading to incompatibility and wasted effort.

1.2 Design Approach

XCS solves this by providing a minimal coordination protocol, a shared namespace for schemas and a standard lifecycle for credentials. The protocol layer handles only what requires global consensus: schema registration, credential lifecycle management, and basic type validation. Everything else remains an application concern: how credentials are stored, verified, presented, secured, and governed.

This separation of concerns is intentional. The protocol provides primitives; applications provide solutions. By keeping the protocol thin, XCS avoids the complexity and ossification that plagues comprehensive standards, while remaining flexible enough to support diverse use cases without modification.

2 Design Philosophy

2.1 Core Principles

The XCS protocol is built on four foundational principles that guide every design decision.

Minimalism shapes protocol scope. The protocol includes only what must be coordinated on-chain. Features are only added if they truly require global consensus across parties who cannot otherwise coordinate offline. This constraint keeps the protocol simple, auditable, and resilient to feature creep.

Composability enables modular design. Schemas can inherit and extend from each other, allowing credential structures to be built from reusable components. This reduces duplication across the ecosystem and enables applications to combine schemas in novel ways without protocol complexity.

Neutrality ensures broad applicability. The protocol has no opinion on credential contents, use cases, storage methods, or verification strategies. It provides infrastructure primitives, not policy constraints. This allows the same protocol to support KYC compliance, gaming achievements, academic credentials, and use cases not yet imagined.

Openness prevents gatekeeping. Anyone can create schemas or issue credentials without permission, gatekeepers, or central authority. No approval process, no vetting, no privileged issuers. This creates a level playing field where innovation comes from applications, not protocol authority.

2.2 Coordination Layer Philosophy

XCS explicitly separates on-chain coordination from off-chain application logic. This distinction is fundamental to the protocol's design.

2.2.1 Protocol Responsibilities (On-chain)

The protocol layer manages shared state that all participants must agree on. This state consists of four key components.

The **schema registry** provides a universal namespace where credential structures are defined and referenced by unique identifiers. Any party can query the registry to discover schema definitions, enabling independent verification of credential formats without coordination with the issuer. This removes the dependency on issuers to provide schema definitions; the protocol becomes the source of truth.

The **credential lifecycle** is tracked using XRPL's native Credentials amendment. The protocol records credential creation, acceptance, and revocation as tamper-proof, cryptographically secured events on-chain. This provides an immutable audit trail without requiring participants to trust any central authority or coordination service.

The **type system** is deliberately basic: strings, integers, booleans, arrays, objects, and addresses. This ensures schema definitions are unambiguous and can be validated consistently across independent implementations. A minimal type system is more important than an expressive one, as it prevents divergent interpretations.

The **schema composition** system enables schemas to extend each other through inheritance. The protocol defines clear rules for how parent schemas are combined, how fields are excluded, and how conflicts are resolved. This determinism ensures all implementations reach identical conclusions about what fields a schema contains.

2.2.2 Application Responsibilities (Off-chain)

Applications own all functionality that does not require global consensus. This boundary between protocol and application is where innovation happens.

Credential storage is entirely application choice. Public credentials can live on IPFS; sensitive data can remain on private servers; hybrid approaches combine both. The protocol specifies how to reference data through URIs, not where or how to store it. This flexibility allows applications to optimize for their specific trust model and security requirements.

Verification logic is implemented by applications according to their use case. KYC applications verify against external identity databases. Gaming applications verify skill achievements and item ownership. Educational applications check institutional records. Privacy-preserving applications implement zero-knowledge proofs. Each domain brings its own requirements; the protocol provides no verification rules.

Privacy and security features are application-implemented. Zero-knowledge proofs, selective disclosure, encryption, access controls, all are built at the application layer. This allows implementations to adapt to varying requirements across jurisdictions and use cases without protocol modification.

Credential presentation is controlled by applications. Metadata standards provide hints; applications decide whether to follow them. Some applications present metadata-rich displays; others implement minimal UI. The protocol provides flexibility; applications provide experience.

Schema governance is a community and organizational concern. Some applications maintain curated schema collections; others trust individual issuers; still others let users validate schemas against their own policies. The protocol imposes no governance model, allowing diverse approaches to flourish.

This separation ensures the protocol remains simple, auditable, and resistant to feature creep while enabling rich, diverse applications. The protocol provides the coordinate system; applications build the cities.

3 Protocol Specification

3.1 Data Types

XCS enforces a basic type system to ensure schemas are unambiguous:

```

1 string    - UTF-8 encoded text
2 uint      - Unsigned integer (0 to 2256-1)
3 int       - Signed integer (-2255 to 2255-1)
4 bytes     - Raw byte array
5 bool      - Boolean (true/false)
6 address   - XRPL address (rBase58 encoded)
7 array<T>  - Ordered list of type T
8 object    - Nested structure

```

Listing 1: Supported Data Types

3.2 Schema Definition

Schemas are the foundation of XCS, defining the structure and types of data that credentials can contain. Each schema acts as a template that issuers reference when creating credentials, ensuring consistent data formats across the ecosystem.

3.2.1 Schema Structure

A schema consists of field definitions mapping names to types:

```

1 {
2   "fields": {
3     "name": "string",
4     "age": "uint",
5     "verified": "bool",
6     "skills": "array<string>",
7     "address": {
8       "street": "string",
9       "city": "string",
10      "postalCode": "uint"
11    }
12  }
13 }

```

Listing 2: Basic Schema Structure

3.2.2 Schema Registration

Creating a schema involves three steps:

1. **Define the schema structure** with required fields and types
2. **Submit registration transaction** via payment with schema in memo
3. **Protocol indexer computes UID** after transaction confirmation

3.2.3 UID Computation

Schema UIDs are computed deterministically by protocol indexers after the transaction confirms on-chain. This design ensures three critical properties.

1. **Determinism** means independent indexers always compute the same UID for the same schema and ledger state.
2. **Uniqueness** is guaranteed because UIDs incorporate issuer address and ledger position, preventing collisions.
3. **Immutability** ensures UIDs cannot be predicted or manipulated before confirmation.

```
1 schema_uid = SHA256(  
2   canonical_json(schema_definition) ||  
3   issuer_address ||  
4   ledger_index ||  
5   transaction_index  
6 )
```

Listing 3: UID Calculation (Deterministic Indexing)

The UID calculation incorporates four components.

1. **Canonical JSON** ensures consistent field ordering regardless of how the schema is serialized.
2. **Issuer address** prevents UID collisions between different issuers creating identical schemas.
3. **Ledger index** records which ledger confirmed the transaction.
4. **Transaction index** captures the position within that ledger.

Since ledger and transaction indices are only known after confirmation, UIDs must be computed by protocol indexers, not issuers. This ensures global uniqueness. Indexers compute UIDs deterministically from ledger state; independent indexers will compute identical results if correctly implemented.

3.2.4 Schema Immutability and Versioning

Schemas are immutable once registered. Versioning works through new schema creation with inheritance:

```
1 // Version 1 - Original schema  
2 {  
3   "fields": {  
4     "name": "string",  
5     "employeeId": "uint"  
6   }  
7 }  
8 // UID: abc123... (computed after registration)  
9  
10 // Version 2 - Extended schema inheriting from v1  
11 {  
12   "extends": ["abc123..."], // Reference v1 by UID  
13   "fields": {  
14     "department": "string", // Add new field  
15     "email": "string"
```

```
16 }
17 }
18 // UID: def456... (new UID computed after registration)
```

Listing 4: Schema Evolution via Inheritance

Each version receives a new UID after registration. Applications choose which schema versions to support, enabling natural migration without breaking existing credentials.

3.3 Schema Inheritance

Inheritance enables modular schema composition and reuse, reducing redundancy across the ecosystem.

3.3.1 Multiple Inheritance

Schemas can inherit from multiple parents, combining their fields:

```
1 // Parent 1: Basic Identity
2 {
3   "uid": "identity_base",
4   "fields": {
5     "name": "string",
6     "dateOfBirth": "uint"
7   }
8 }
9
10 // Parent 2: Contact Info
11 {
12   "uid": "contact_base",
13   "fields": {
14     "email": "string",
15     "phone": "string"
16   }
17 }
18
19 // Child: Employee combining both parents
20 {
21   "extends": ["identity_base", "contact_base"],
22   "fields": {
23     "employeeId": "uint",
24     "department": "string"
25   }
26 }
27 // Result: name, dateOfBirth, email, phone, employeeId, department
```

Listing 5: Multiple Inheritance Example

3.3.2 Selective Field Exclusion

The `exclude` mechanism allows fine-grained control over inherited fields:

```
1 {
2   "extends": ["parent_uid_1", "parent_uid_2"],
3   "exclude": {
4     "parent_uid_1": ["sensitiveField"],
5     "parent_uid_2": ["unnecessaryField"]
6   },
7   "fields": {
```

```

8     "newField": "string"
9   }
10 }

```

Listing 6: Selective Field Exclusion

3.3.3 Inheritance Rules

The protocol enforces clear inheritance semantics to ensure deterministic schema composition. When processing a schema with multiple parents, potential conflicts can arise from duplicated fields, child redefinitions, or exclusions. The protocol resolves these through a deterministic four-step algorithm.

1. **Exclusions are applied first.** Any fields specified in the exclusion list are removed from parent schemas before conflict analysis. This gives exclusions highest priority.
2. **Remaining conflicts are detected.** If the same field name exists in multiple parents after exclusion, the schema is flagged invalid. This prevents non-deterministic behavior where the order of parent schemas would determine which field applies.
3. **Child schemas can override inherited fields.** A field defined in the current schema replaces any inherited field with the same name, regardless of parent order. This allows child schemas to redefine inherited behavior when needed.
4. **Validation marks schemas with unresolved conflicts.** If any conflicts remain after applying these three rules, the indexer marks the schema invalid. An invalid schema cannot be used for creating credentials that comply with the XCS protocol. While any party can submit a transaction on-chain (the protocol cannot prevent this), only credentials referencing valid schemas can be recognized as compliant with XCS. Applications will refuse to verify credentials against invalid schemas, making them effectively unusable for any credential that needs protocol-compliant verification. This strict approach prevents ambiguous credential structures and ensures all valid XCS credentials have unambiguous structure.

Consider three examples illustrating these rules:

```

1 // Invalid: Unresolved conflict
2 {
3   "extends": ["schema_a", "schema_b"], // Both define "id"
4   "fields": { "status": "string" }
5 }
6 // Result: INVALID - both parents define "id", no resolution
7
8 // Valid: Resolved via exclusion
9 {
10  "extends": ["schema_a", "schema_b"],
11  "exclude": {
12    "schema_b": ["id"] // Remove conflicting field
13  },
14  "fields": { "status": "string" }
15 }
16 // Result: Valid - uses schema_a's "id", schema_b's "id" excluded
17
18 // Valid: Resolved via child override
19 {
20  "extends": ["schema_a", "schema_b"],

```



```

21  "fields": {
22    "id": "uint"  // Child redefines the field
23  }
24 }
25 // Result: Valid - uses child's "id" definition

```

Listing 7: Inheritance Conflict Resolution

This inheritance system enables composition while maintaining determinism and preventing ambiguity. Indexers validate inheritance during processing; only valid schemas are available for credential creation.

3.4 Protocol Operations

XCS leverages the XRPL Credentials amendment for credential lifecycle while adding a schema coordination layer through payment transactions with specially formatted memos.

3.4.1 Two-Layer Architecture

XCS operates at two distinct layers:

Schema Layer: Uses XRPL payment transactions with memo field to register and discover schemas. The memo carries schema definitions, prefixed with `xcs:` for namespace isolation.

Credential Layer: Directly uses XRPL's native Credentials amendment (`CredentialCreate`, `CredentialAccept`, `CredentialDelete`) without modification, referencing schemas by UID.

This design means XCS requires no changes to XRPL itself; it only adds coordination on top of existing primitives.

3.4.2 XCS Core Operations and Memo Prefixing

All XCS protocol operations use the `xcs:` prefix in transaction memo fields to ensure namespace isolation and enable efficient indexing. This prefix allows XCS indexers to quickly identify protocol-relevant transactions without parsing unrelated XRPL activity, while also providing a clear boundary between XCS activity and general ledger use. The convention also supports extensibility, allowing future operations (e.g., `xcs:schema_deprecate`) to be introduced unambiguously.

The protocol defines four core memo types corresponding to credential lifecycle operations:

```

1 xcs:schema_register - Register a new schema
2 xcs:credential_create - Create and issue a credential
3 xcs:credential_accept - Accept an issued credential
4 xcs:credential_revoke - Revoke an existing credential

```

Listing 8: XCS Memo Types

Each memo type maps directly to a core XRPL operation. Schema registration defines new immutable credential schemas. Once confirmed on-ledger, schema UIDs are computed and become globally unique. Credential creation issues credentials based on registered schemas using the native `CredentialCreate` transaction. Credential acceptance allows credential subjects to accept or reject credentials via `CredentialAccept`. Credential revocation enables issuers to revoke credentials using `CredentialDelete`.

Only the first two operations (schema registration and credential creation) require memo tracking for protocol indexing. Schema registration must be logged to compute post-confirmation schema UIDs, and credential creation must be recorded to establish the initial on-chain reference. Acceptance and revocation events are optional in memos, as their state can be fully reconstructed

from native XRPL credential entries. Nonetheless, applications may include these memos to embed contextual data such as revocation reasons or acceptance metadata for richer off-chain indexing or analytics.

Schemas are immutable once registered. Any updates require creating a new schema, optionally inheriting structure from prior versions, ensuring reproducibility and clear version lineage.

3.4.3 Schema Registration

Creates a new schema in the protocol registry:

```
1 Transaction Type: Payment
2 Destination: SCHEMA_REGISTRY_ADDRESS
3 Amount: SCHEMA_REGISTRATION_FEE XRP
4 Memo:
5   Type: "xcs:schema_register"
6   Data: {
7     "extends": [optional_parent_uids],
8     "exclude": {optional_exclusions},
9     "fields": {field_definitions}
10  }
```

Listing 9: Schema Registration Transaction

3.4.4 Credential Issuance

Issues a credential conforming to a registered schema:

```
1 Transaction Type: CredentialCreate
2 Fields:
3   Subject: recipient_address
4   CredentialType: schema_uid
5   URI: storage_location
6   Expiration: optional_unix_timestamp
7   Memo:
8     Type: "xcs:credential_create"
```

Listing 10: Credential Creation

3.4.5 Credential Acceptance

Accepts an issued credential:

```
1 Transaction Type: CredentialAccept
2 Fields:
3   Issuer: issuer_address
4   CredentialType: schema_uid
```

Listing 11: Credential Acceptance

3.4.6 Credential Revocation

Revokes an existing credential:

```
1 Transaction Type: CredentialDelete
2 Fields:
3   Subject: subject_address
4   Issuer: issuer_address
5   CredentialType: schema_uid
```

Listing 12: Credential Deletion

3.5 Credential Metadata Standard

While schemas define data structure, applications need additional information for discovery, presentation, and user experience. The protocol supports an optional metadata layer in off-chain storage alongside credential data.

3.5.1 Metadata Purpose

Metadata serves application needs without requiring protocol coordination. **Discovery** helps users find and understand credential types, enabling search and browsing of available schemas. **Presentation** provides display hints through logos, colors, and icons that applications can use to create consistent user experiences. **Context** supplies issuer information and credential descriptions so users understand what credentials mean and who issued them. **Localization** enables applications to provide multi-language support for credentials, making them accessible to global audiences.

3.5.2 Metadata Structure

Credential metadata follows a standardized but extensible format:

```
1 {
2   "schemaVersion": "credential.v0",
3   "issuer": {
4     "name": "Acme University",
5     "logo": "ipfs://QmYyy...",
6     "url": "https://acme.edu"
7   },
8   "credential": {
9     "name": "Bachelor of Science",
10    "description": "Computer Science Degree",
11    "image": "ipfs://QmZzz...",
12    "backgroundColor": "#1a73e8",
13    "textColor": "#ffffff"
14  },
15  "display": {
16    "card": {
17      "logo": "ipfs://QmAaa...",
18      "backgroundColor": "#ffffff",
19      "textColor": "#000000"
20    },
21    "icon": "ipfs://QmBbb..."
22  },
23  "attributes": [
24    {
25      "field": "studentName",
26      "label": "Student Name",
27      "display": true
28    },
29    {
30      "field": "gpa",
31      "label": "GPA",
32      "display": false
33    }
34  ]
}
```

35 }

Listing 13: Credential Metadata Structure

3.5.3 Metadata Application

The protocol maintains clear separation of concerns across three layers. The **schema** defines data structure and is stored on-chain as an immutable reference. **Credential data** contains the actual values conforming to the schema and is managed off-chain by applications according to their storage strategy. **Metadata** provides presentation hints and discovery information, also managed off-chain by applications.

Applications choose whether to include metadata, what information to provide, and how strictly to follow the metadata hints. This flexibility enables diverse use cases without protocol constraints.

3.6 XCS Indexer

The XCS indexer is an off-chain component that observes XRPL transactions and constructs a queryable view of credential and schema state. Because all protocol-relevant data is recorded on-chain, indexers require no independent consensus and can operate deterministically from public ledger data.

3.6.1 Responsibilities

An indexer performs the following tasks:

- **Transaction Monitoring:** Listen to XRPL ledgers for transactions whose memo fields begin with the `xcs:` prefix.
- **Schema Reconstruction:** Extract and validate schema definitions, compute deterministic UIDs, and record inheritance relationships.
- **Credential State Tracking:** Map credential lifecycle events (`CredentialCreate`, `CredentialAccept`, `CredentialDelete`) to their associated schema UIDs and participating addresses.
- **Validation and Conflict Detection:** Ensure that inheritance rules and type definitions resolve deterministically; flag invalid schemas.
- **Queryable Interface (Optional):** Provide applications with structured access to schema and credential metadata via local APIs or database queries.

3.6.2 Deterministic Output

Indexers must produce identical outputs given the same ledger state. To guarantee reproducibility, all indexers:

- Use canonical JSON encoding when hashing schema definitions.
- Apply ledger index + transaction index ordering when computing UIDs.
- Maintain a monotonic ledger replay model, processing transactions strictly by ledger sequence.

3.7 Cryptographic and Trust Model

3.7.1 Inherited Security from XRPL

XCS inherits the cryptographic guarantees and trust assumptions of the XRP Ledger. The protocol assumes:

- **Ledger Finality:** Once a transaction is validated by the XRPL consensus process, it is immutable and globally ordered. All schema registrations and credential lifecycle events depend on this property for deterministic indexing and verifiable history.
- **Cryptographic Signatures:** Each XRPL transaction is signed by the private key controlling the sending account. Credential issuers are therefore provably in possession of the private key corresponding to the `issuer_address`. This ensures non-repudiation: only the legitimate holder of that key can issue, revoke, or modify credentials under that address.
- **Key Ownership and Authentication:** Trust in an issuer derives from XRPL account-level cryptographic control. Applications may layer additional trust frameworks (e.g., registry whitelists or reputation systems), but the base protocol only authenticates via ledger-verified signatures.

3.7.2 XCS-Specific Guarantees

Beyond XRPL's guarantees, XCS adds the following deterministic coordination properties:

- **Schema UID Determinism:** Schema identifiers are computed via canonical JSON hashing and ledger state parameters. Independent indexers computing from the same ledger state will derive identical results, ensuring global reproducibility.
- **Schema and Credential Immutability:** Once registered, schemas cannot be modified; new versions must inherit from previous ones. Similarly, credential lifecycle events (create, accept, revoke) are immutable transactions recorded on-chain.
- **Protocol Coordination Layer:** XCS introduces no new consensus mechanism. It relies entirely on XRPL's finality guarantees while defining how credential-related transactions are interpreted and indexed. This design ensures auditability without expanding the system's trust surface.

3.8 Credential Storage and Verification

The protocol does not mandate storage strategies. Common patterns that applications employ include:

Public Credentials:

XRPL Entry → IPFS Content → Public Verification

Private Credentials:

XRPL Entry → Issuer API → Authorized Verification

3.8.1 Verification Flow

A typical verification flow involves four sequential steps. First, applications retrieve the credential from XRPL by querying for entries matching the subject, issuer, and credential type. During this step, the application checks whether the credential has been accepted by the subject and whether it has expired.

Second, applications fetch the actual credential data from storage. For public credentials stored on IPFS or similar systems, retrieval is straightforward. For private credentials stored on issuer APIs, the application must provide appropriate authorization credentials.

Third, applications validate the credential data against its schema. This involves retrieving the schema from the registry, recursively resolving any parent schemas to get the complete field set, and type-checking each credential field.

Fourth, applications perform use-case-specific verification. This is where business logic is applied, privacy-preserving proofs are checked, or external data sources are queried. Each application determines what additional verification is needed for its context.

See the Appendix for verification pseudocode showing this flow in detail.

4 Illustrative Application

XCS enables diverse applications across multiple domains. The following example demonstrate the framework's versatility.

4.1 XRP Name Service: Decentralized Profiles

The XRP Name Service demonstrates how XCS enables higher-level applications. Developers can use XNS credentials to build a decentralized naming system where profiles are verifiable data structures:

```
1 {  
2   "fields": {  
3     "name": "string",  
4     "address": "address",  
5     "avatar": "string",  
6     "bio": "string",  
7     "social": "array<string>",  
8     "subdomains": "array<object>"  
9   }  
10 }
```

Listing 14: XRP Name Service Schema

This enables several key capabilities. With **human-readable addressing**, users send funds to `alice.xrp` instead of lengthy hex addresses, dramatically improving usability and reducing transaction errors from address mistyping. **Decentralized profiles** store user profile information as verifiable credentials, giving users complete control and preventing the platform lock-in common in Web2 identity systems. **Hierarchical namespaces** use subdomains to enable organization-specific naming hierarchies like `alice.gaming.xrp`, allowing specialized use cases while maintaining a unified namespace on the XRPL. This exemplifies how XCS enables practical infrastructure improvements that enhance the entire ecosystem while remaining fully decentralized and user-controlled.

5 Security Considerations

This section describes security concerns specific to the XCS protocol layer and how applications should address them. Security is primarily an application responsibility.

5.1 Schema Poisoning

Malicious actors can create credentials that claim to conform to reputable schemas but actually reference their own fraudulent schemas with similar names. This is an application governance problem, not a protocol vulnerability.

Applications should maintain trusted schema registries organized by issuer, ensuring they only accept schemas from known, reputable sources. When accepting a schema, applications should verify schema UUIDs rather than relying on schema names, since different issuers can create schemas with identical names but different UUIDs. During verification, applications should display the full schema UUID in results so users can independently verify they are checking against the intended schema. User education is critical, applications should help users understand the importance of schema UUID verification and warn them about accepting schemas from unknown issuers.

5.2 Storage Availability

IPFS content may become unavailable if nodes stop pinning the data. This is an application storage problem, not a protocol issue.

Applications should employ multiple pinning services for critical credentials, ensuring data persistence even if individual pinning nodes go offline. Implementing fallback to issuer-hosted storage provides a backup when IPFS becomes unavailable. For important credentials, applications should maintain local backups so users retain access even if external storage becomes inaccessible. For truly critical data, applications should require issuer-hosted guarantees committing to maintaining availability for specific periods or accepting liability for data loss.

5.3 Privacy and Relationship Disclosure

On-chain credential existence reveals social relationships—observers can see that X issued a credential to Y, potentially inferring relationships that should remain private. This is an application privacy problem.

Applications can use separate accounts for different credential contexts, preventing correlation of credentials issued to the same account. Batching credential operations obscures timing patterns that could otherwise reveal the temporal relationship between credential issuance and life events. Implementing private credentials for sensitive relationships keeps the fact of relationship off-chain. Applications can employ privacy-preserving verification techniques that prove facts about credentials without revealing the credentials themselves or even their existence.

5.4 Type Validation

Incorrect type validation during schema creation or credential verification could enable attacks where malicious actors create credentials that appear valid but contain data in unexpected formats. This is an application implementation concern.

Applications should implement strict type checking at all verification layers, rejecting any credential data that does not conform exactly to the schema's specified types. Using canonical JSON representation for schema hashing ensures consistent hashing regardless of serialization differences. Validating schema definitions before registration prevents acceptance of malformed

schemas. Applications should test type validation against edge cases like extremely large numbers, deeply nested structures, and unusual characters in strings, ensuring robustness against adversarial inputs.

6 Future Work

6.1 Protocol Extensibility

The protocol’s design enables future extensions without breaking changes. One potential extension is `xcs:schema_deprecate`, which would allow marking schemas as deprecated while maintaining backward compatibility with existing credentials. Another is `xcs:credential_suspend`, enabling temporary credential suspension as distinct from permanent revocation. A third possibility is `xcs:delegation`, allowing issuers to delegate issuance authority to other addresses. Beyond these protocol extensions, domain-specific extensions can be built on top of core primitives—applications can define specialized credential types, metadata formats, and verification procedures suited to their specific needs.

All extensions can be added independently without modifying the core protocol, as long as they use distinct memo prefixes for namespace isolation. This design approach ensures the core protocol remains stable and auditable while enabling innovation at the edges.

6.2 Ecosystem Development

Development of supporting infrastructure and tools can accelerate adoption and reduce barriers to entry. Software Development Kits (SDKs) for major programming languages would provide libraries for credential creation, issuance, and verification, enabling developers to integrate XCS without building low-level infrastructure. A Schema Repository would provide discoverable, curated collections of well-designed schemas, helping applications reuse proven patterns rather than reinventing credential structures. Third-party Verification Services tailored to different domains—KYC, education, gaming, and others—would provide specialized verification capabilities without requiring each application to build this expertise. Privacy Layer libraries implementing zero-knowledge proofs and selective disclosure would enable applications to offer privacy-preserving verification without cryptographic expertise.

7 Conclusion

XCS establishes a minimal protocol for coordinating credential schemas and lifecycle events on the XRP Ledger. Its design is intentionally narrow: the protocol defines what must be agreed upon globally and nothing more. This constraint ensures that implementations remain interoperable, auditable, and adaptable as the ecosystem evolves.

The result is a foundation rather than a framework—one that applications can extend freely without protocol changes. Future work will formalize technical interfaces and implementation details to enable full interoperability across independent XCS deployments.

References

TODO ADD REFERENCES

- [1] XLS-70d-OnChain-Credentials <https://xls.xrpl.org/xls/XLS-0070-credentials.html>
- [2] XRP Ledger Protocol Documentation

[3] XLS-24-NFT-Metadata <https://xls.xrpl.org/xls/XLS-0024-nft-metadata.html>

A Verification Pseudocode

```

1 def verify_credential(subject_address, issuer_address, schema_uid):
2     """
3     Verify a credential exists and conforms to its schema.
4     This is a reference implementation; applications may add
5     additional verification logic specific to their use case.
6     """
7
8     # Step 1: Retrieve credential lifecycle from XRPL
9     credential = xrpl_query(
10         subject=subject_address,
11         issuer=issuer_address,
12         credential_type=schema_uid
13     )
14
15     if not credential:
16         return VerificationError("Credential not found on-chain")
17
18     if not credential.accepted:
19         return VerificationError("Credential not accepted by subject")
20
21     if credential.expiration and credential.expiration < current_time():
22         return VerificationError("Credential has expired")
23
24     # Step 2: Fetch credential data from storage
25     if credential.uri.startswith("ipfs://"):
26         # Public credential on IPFS
27         credential_data = ipfs_fetch(credential.uri)
28     elif credential.uri.startswith("https://"):
29         # Private credential from issuer API
30         auth_token = generate_auth_proof(subject_address)
31         credential_data = api_fetch(
32             url=credential.uri,
33             headers={"Authorization": auth_token}
34         )
35     else:
36         return VerificationError("Unsupported storage protocol in URI")
37
38     # Step 3: Validate credential data against schema
39     schema = fetch_schema(schema_uid)
40
41     if not schema:
42         return VerificationError("Schema not found in registry")
43
44     if schema.invalid:
45         return VerificationError("Schema has validation errors")
46
47     # Recursively resolve parent schemas to get full field set
48     full_schema = resolve_inheritance(schema)
49
50     # Type-check all credential fields against resolved schema
51     for field, field_type in full_schema.items():
52         if field in credential_data:
53             if not validate_type(credential_data[field], field_type):
54                 return VerificationError(
55                     f"Type mismatch for field '{field}': "

```

```

56         f"expected {field_type}, got {type(credential_data[field])}"
57     )
58
59     # Step 4: Application-specific verification
60     # This is where applications add domain-specific logic
61
62     if custom_verification_required():
63         # Application can implement business rule validation
64         if not validate_business_logic(credential_data):
65             return VerificationError("Business rule validation failed")
66
67         # Application can verify zero-knowledge proofs
68         if "zkp_proof" in credential_data:
69             if not verify_zkp(credential_data["zkp_proof"]):
70                 return VerificationError("Zero-knowledge proof verification
71 failed")
72
73         # Application can correlate with external data sources
74         if requires_external_validation():
75             external_valid = check_external_sources(credential_data)
76             if not external_valid:
77                 return VerificationError("External validation failed")
78
79     return VerificationSuccess(credential_data)
80
81 def resolve_inheritance(schema):
82     """
83     Recursively resolve schema inheritance chain.
84     Returns a flat dictionary of field names to types.
85     """
86     resolved_fields = {}
87
88     # Process parent schemas if inheritance is defined
89     if schema.extends:
90         for parent_uid in schema.extends:
91             try:
92                 parent = fetch_schema(parent_uid)
93             except:
94                 raise SchemaResolutionError(
95                     f"Parent schema {parent_uid} not found"
96                 )
97
98             # Recursively resolve parent's inheritance
99             parent_fields = resolve_inheritance(parent)
100
101             # Apply exclusion rules
102             if parent_uid in schema.exclude:
103                 for excluded_field in schema.exclude[parent_uid]:
104                     parent_fields.pop(excluded_field, None)
105
106             # Merge parent fields and detect conflicts
107             for field, field_type in parent_fields.items():
108                 if field in resolved_fields:
109                     if resolved_fields[field] != field_type:
110                         raise SchemaConflictError(
111                             f"Field conflict for '{field}': "
112                             f"received {field_type} but already have "
113                             f"{resolved_fields[field]}"
114                         )
115                     else:
116                         resolved_fields[field] = field_type

```

```
116
117     # Add current schema's fields (override parents)
118     resolved_fields.update(schema.fields)
119
120     return resolved_fields
```

Listing 15: Credential Verification Pseudocode

B Example Schema Definitions

B.1 Simple KYC Credential

```
1 {
2   "fields": {
3     "verified": "bool",
4     "verificationLevel": "uint",
5     "country": "string",
6     "verificationDate": "uint",
7     "expiryDate": "uint"
8   }
9 }
```

Listing 16: KYC Verification Schema

B.2 Composable Identity Schema

Base Identity:

```
1 {
2   "uid": "base_identity_xyz",
3   "fields": {
4     "fullName": "string",
5     "dateOfBirth": "uint",
6     "nationality": "string"
7   }
8 }
```

Listing 17: Base Identity Schema

Extended with Employment:

```
1 {
2   "extends": ["base_identity_xyz"],
3   "fields": {
4     "employeeId": "uint",
5     "department": "string",
6     "employmentDate": "uint"
7   }
8 }
```

Listing 18: Employment Extension

B.3 Gaming Achievement Credential

```
1 {
2   "fields": {
3     "achievement": "string",
```

```
4     "rarity": "uint",  
5     "unlockedDate": "uint",  
6     "gameVersion": "string",  
7     "verificationMethod": "string"  
8   }  
9 }
```

Listing 19: Gaming Achievement Schema